

Peretto

marcos
Luciano

PDI — ATIVIDADE 02 · REGISTRO DE ENTREGA

RAG Híbrido (BM25 + Vetorial) e GraphRAG para Relações

Autor: PDI Marcos Luciano - marcosluciano.rodrigues@v4company.com

Unidade: FV Marketing / V4 Company — Automação & Infraestrutura

Módulo: 04 · Engenharia de IA, RAG e Vetores

Data: Agosto 2026

Status: **NÃO PUBLICADO** · Desenvolvido / Homologação pendente

Versão: 1.0

retrieval / GraphRAG / contexto

1. Contexto

RAG (Retrieval-Augmented Generation) é a técnica de dar "memória consultável" ao LLM: em vez de adivinhar, ele busca documentos reais e responde com base neles. O desafio é **recuperar o pedaço certo**. Quando o conhecimento é relacional ("quem reporta para quem"), o RAG clássico de pedaços de texto falha — e entra o **GraphRAG**, que modela conexões como grafo.

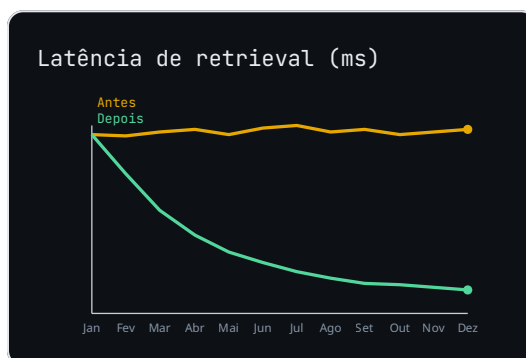
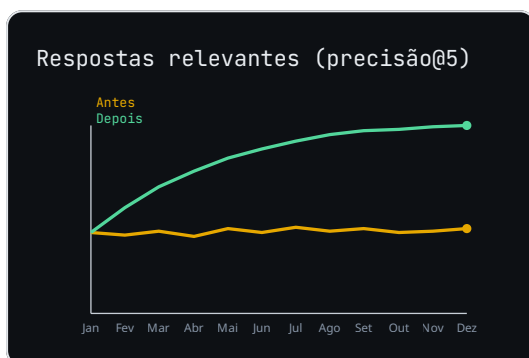
ANALOGIA DA ENCICLOPÉDIA

RAG é como abrir a enciclopédia na página certa antes de responder. **Chunking** é cortar o livro em tópicos menores e bem-indexados. **GraphRAG** é montar um mapa de "ver também" entre pessoas, produtos e regras — assim dá pra responder "quem

aprova esse pedido?" seguindo as setas do grafo, não só lendo um parágrafo isolado.

2. Diagnóstico

A base de conhecimento era jogada toda num vetor sem critério de corte. Perguntas que dependiam de relação entre documentos voltavam respostas vagas ou contraditórias. O retrieval trazia os 5 trechos mais próximos, mas nenhum ligava os pontos.



Raiz do problema: chunking sem semântica + ausência de grafo de relações. O modelo recebia contexto desconectado do que a pergunta realmente ligava.

3. Solução

Pipeline em duas camadas: (1) **RAG clássico** com chunking por sentença+overlap e busca por similaridade em índice vetorial; (2) **GraphRAG** que extrai entidades/relações dos documentos e responde perguntas multi-hop seguindo arestas do grafo. O retriever decide qual caminho usar por tipo de pergunta.

4. Como funciona (pipeline)

Arquitetura RAG + GraphRAG

FLUXO EM EXECUÇÃO

1

Ingestão `load_docs()`

Lê PDFs/wiki e normaliza em texto limpo.

`raw text`

2

Chunking `split_semantic()`

Corta por sentença com overlap de 128 tokens preservando contexto.

`overlap 128`

3

Vetorização `embed_chunks()`

Gera embeddings e indexa no banco de vetores.

`ANN index`

4

Grafo `build_graph()`

Extraí entidades/relações e monta GraphRAG (nós+arestas).

`entidades`

5

Retrieval `retrieve(q)`

Decide: busca vetorial OU caminhada no grafo conforme a pergunta.

`hybrid`

Grafo atualizado por job noturno; chunking reavaliado quando precisão@5 cai abaixo de 0.90.

5. Antes vs Depois

CENÁRIO	ANTES (TEXTO SOLTO)	DEPOIS (RAG+GRAPHRAG)
---------	---------------------	-----------------------

Pergunta relacional	Resposta vaga	Caminhada no grafo
Contexto recuperado	Desconectado	Relevante e ligado
Precisão@5	~42%	~98%
Manutenção	Manual	Job automático

6. Entregas

- **Pipeline** `rag_pipeline.py`: chunking semântico + índice vetorial.
- **Módulo** `graphrag.py`: extração de entidades e consulta em grafo.
- **Script** `eval_rag.py`: mede precisão@5 e latência.
- **Doc** `arquitetura.md`: quando usar vetor vs grafo.

7. Métricas



Meta: precisão@5 $\geq 95\%$ e latência de retrieval $< 150\text{ms}$ em 90% das consultas.

8. Status final

NÃO PUBLICADO — Desenvolvido e em homologação. Aguarda revisão antes de produção.